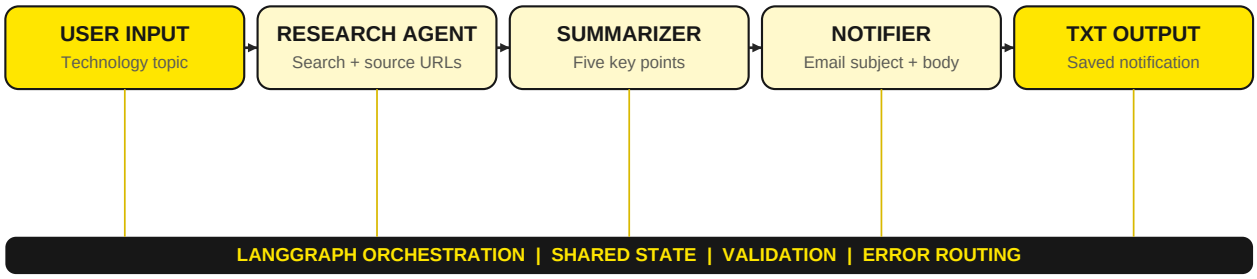
AGILE PROJECT DOCUMENTATION / EPIC BASELINE

Multi-Agent AI News Assistant

A structured project definition for research, summarization, employee notification, and LangGraph integration.

EPIC ID	**EPIC-01**	**PROJECT TYPE**	Generative AI / Multi-Agent System
LANGUAGE	**Python**	**ORCHESTRATION**	LangGraph
TEAM	**3 developers**	**LLM ACCESS**	OpenRouter API (proposed)

End-to-end agent workflow



PROJECT OUTCOME

The user enters a technology topic. The Research Agent gathers recent web findings; the Summarizer produces five concise, source-backed points; and the Notifier generates a professional employee email saved as a .txt file. LangGraph coordinates the workflow and its failure paths.

Prepared for Agile planning, team execution, and project review.

DOCUMENT GUIDE

SECTION	CONTENTS
01	Epic overview and problem statement alignment
02	Scope, objectives, assumptions, and exclusions
03	Feature breakdown and ownership
04	Eight user stories, acceptance criteria, and tasks
05	Agent input/output contracts
06	Architecture, LangGraph orchestration, and state
07	Non-functional requirements and project structure
08	Sprint plan, test scenarios, and Definition of Done
09	Ownership, Agile board, and final success criteria

01 / EPIC OVERVIEW

Epic title: Develop a Multi-Agent AI News Assistant

Epic description: As an employee, I want an AI-powered multi-agent assistant that automatically researches the latest developments in a user-provided technology topic, summarizes the research into five concise and informative points with source links, and generates a professional employee notification email saved as a text file, so that I can stay updated about emerging technologies without manually searching multiple sources.

Problem statement alignment

The project addresses the effort involved in manually discovering, reviewing, summarizing, and communicating technology updates. It automates the process through three specialized agents—web research, AI summarization, and email notification—coordinated as one workflow. The final deliverable is an employee notification saved as a plain-text file.

Business value

- Reduces time spent manually researching emerging technologies.
- Converts multiple research findings into five concise takeaways.
- Preserves source links so readers can verify information.
- Automates preparation of a consistent employee notification.
- Demonstrates modular agent development and LangGraph orchestration.

Epic goal

Deliver an end-to-end application that accepts a topic, gathers relevant recent web information, generates exactly five grounded key points when sufficient evidence exists, creates a professional email, and saves it as a .txt file.

02 / SCOPE AND REQUIREMENTS

In scope

- Accept and validate a user-provided technology topic.
- Search the web using a search API/tool and collect multiple relevant sources.
- Capture titles, descriptions, source URLs, and publication dates where available.
- Summarize findings into exactly five concise points when evidence is sufficient.
- Associate source references with relevant summary points.
- Generate a professional email subject and body containing all five points and links.
- Save the email as a .txt file in a designated output directory.
- Coordinate agents, state, and error handling using LangGraph.

Out of scope for the initial release

- Automatically sending email through an email provider; the requirement is to generate and save a text file.
- User accounts, employee directory management, and role-based access control.
- A production dashboard or scheduled news delivery unless added later.
- Guaranteeing every web result is correct; the system must preserve sources and avoid unsupported claims.

Assumptions and dependencies

- A web search API/tool and LLM endpoint will be available with valid credentials.
- Python and LangGraph are used for the initial implementation.
- OpenRouter is a proposed LLM provider; a compatible alternative may be selected.
- The team agrees on shared input/output contracts before integration.
- Publication dates are optional because they may not be supplied by every source.

03 / FEATURE BREAKDOWN AND OWNERSHIP

ID	FEATURE	PURPOSE	OWNER
F-01	**Web Research Agent**	Search recent technology developments and return structured findings with source URLs.	Sreedev CS
F-02	**AI Summarization Agent**	Analyze findings and produce five concise, grounded points with references.	Vishal Kumar Lal
F-03	**Email Notification Agent**	Generate a professional employee email and save it as a .txt file.	Vivek Sharma
F-04	**Multi-Agent Orchestration & Integration**	Coordinate agents using LangGraph, manage shared state, sequence execution, and handle errors.	All members

F-01 — Web Research Agent

Owner: Sreedev CS

Accept a validated topic, search the web, collect relevant information from multiple sources, and pass structured findings to the Summarization Agent.

Expected output: Topic, findings, source URLs, optional publication dates, status, and error details.

F-02 — AI Summarization Agent

Owner: Vishal Kumar Lal

Analyze structured findings and produce exactly five concise, professionally written points when sufficient valid research is available. Retain appropriate source references.

Expected output: Topic, five key points, associated source links, status, and error details.

F-03 — Email Notification Agent

Owner: Vivek Sharma

Convert the five-point summary into a professional email with subject, introduction, key points, source links, and closing; save it as a .txt file.

Expected output: Email subject, email body, output file path, status, and error details.

F-04 — Multi-Agent Orchestration & Integration

Owner: Shared — all three members

Integrate the independently developed agents using LangGraph. Define shared state, connect nodes in the required order, validate outputs, and handle failures.

Expected output: A working end-to-end workflow that creates the notification file when all stages succeed.

04 / USER STORIES, ACCEPTANCE CRITERIA AND TASKS

Acceptance criteria are measurable conditions for accepting a story. Tasks are the implementation activities needed to deliver it.

US-01 | Accept Technology Topic

Owner: Sreedev CS

User story: As an employee, I want to enter a technology topic so that the AI assistant can research the latest developments related to it.

Acceptance criteria

- The system accepts a topic through user input.
- Empty or whitespace-only topics are rejected.
- A valid topic is passed to the Research Agent.
- Invalid input produces an appropriate error message.
- Relevant topics such as Generative AI, Agentic AI, Cloud Computing, and Cybersecurity can be entered.

Implementation tasks

- T-01: Create the user input mechanism.
- T-02: Implement input validation.
- T-03: Pass the validated topic to the Research Agent.

US-02 | Search Latest Technology Developments

Owner: Sreedev CS

User story: As a user, I want the Research Agent to search the web so that I receive relevant and recent information about the selected technology.

Acceptance criteria

- The agent performs web searches using a search API/tool.
- Information is collected from multiple relevant sources.
- Each finding includes a title, description, and source URL.
- Publication dates are captured wherever available.
- Results prioritize recent developments.
- Search failures are handled gracefully.
- Empty results are reported; the agent does not invent findings.
- Structured research is passed to the Summarizer Agent.

Implementation tasks

- T-04: Select and configure a web search API/tool.
- T-05: Implement search functionality.
- T-06: Retrieve and extract relevant search results.
- T-07: Extract source URLs and available publication dates.
- T-08: Filter irrelevant or duplicate findings.
- T-09: Structure research output.
- T-10: Implement search error handling.

US-03 | Summarize Research Findings

Owner: Vishal Kumar Lal

User story: As an employee, I want the AI assistant to summarize research findings into five concise points so that I can quickly understand the most important technology developments.

Acceptance criteria

- The Summarizer Agent receives structured research findings.
- The agent analyzes the available information.
- It generates exactly five key points when sufficient valid research exists.
- Each point is concise, clear, and professionally written.
- The points represent important findings from the research.
- Repetition is avoided.
- Unsupported information is not introduced.
- Empty or insufficient research does not produce fabricated content.

Implementation tasks

- T-11: Configure the LLM API and model.
- T-12: Create the Summarizer Agent.
- T-13: Design the summarization prompt.
- T-14: Process and analyze research output.
- T-15: Generate five key points.
- T-16: Validate summary structure and point count.
- T-17: Handle empty, incomplete, or invalid research input.

US-04 | Include Source References in Summary

Owner: Vishal Kumar Lal

User story: As an employee, I want source links included with the summarized information so that I can verify the original technology developments.

Acceptance criteria

- Each summary point includes relevant source URLs or clearly identified references.
- Links correspond to original research findings.
- Duplicate source URLs are avoided.
- Missing or invalid URLs are not presented as verified references.
- Source information is preserved when passed to the Notifier Agent.

Implementation tasks

- T-18: Map research sources to summary points.
- T-19: Validate source URLs.
- T-20: Remove duplicate references.
- T-21: Structure summary and references for downstream processing.
- T-22: Test source attribution and summary accuracy.

US-05 | Generate Employee Notification Email

Owner: Vivek Sharma

User story: As an employee, I want a professionally written technology update email so that I can stay informed about the latest industry developments.

Acceptance criteria

- The Notifier Agent receives the validated summary.
- A professional subject line is generated.
- The email contains an appropriate introduction.
- The body includes all five key points.
- Relevant source links are included.
- A professional closing statement is present.
- The email is readable and properly formatted.

Implementation tasks

- T-23: Create the Email Notification Agent.
- T-24: Design the email-generation prompt.
- T-25: Generate a suitable subject line.
- T-26: Generate the email body from the summary.
- T-27: Include all five points and references.
- T-28: Validate email structure and formatting.

US-06 | Save Email Notification as Text File

Owner: Vivek Sharma

User story: As an administrator, I want the generated employee notification to be saved as a text file so that it can be accessed and reviewed later.

Acceptance criteria

- The email is saved in .txt format.
- The file contains the subject and complete email body.
- All five points and source links are preserved.
- The file is saved in the designated output directory.
- The filename is meaningful and identifiable.
- Successful file creation is confirmed.
- File-writing and directory errors are handled.

Implementation tasks

- T-29: Create the output directory if it does not exist.
- T-30: Implement text-file generation.
- T-31: Save the email subject and body.
- T-32: Implement a meaningful filename convention.
- T-33: Handle file-system errors.
- T-34: Verify generated file contents.

US-07 | Coordinate Three AI Agents

Owner: Shared — all members

User story: As a developer, I want all three AI agents coordinated using LangGraph so that the complete research-to-notification workflow executes automatically.

Acceptance criteria

- Three independent agents are implemented.
- LangGraph coordinates their execution.
- A shared state structure manages workflow data.
- Research output is passed to the Summarizer Agent.
- Summary output is passed to the Notifier Agent.
- Agents execute in the correct sequence.
- File generation occurs only after valid preceding outputs.
- Agent failures are propagated and handled.

Implementation tasks

- T-35: Define the shared LangGraph state schema.
- T-36: Create the Research Agent node.
- T-37: Create the Summarizer Agent node.
- T-38: Create the Notifier Agent node.
- T-39: Define graph edges and execution sequence.
- T-40: Configure graph entry point and termination.
- T-41: Integrate individual agent modules.
- T-42: Test state transfer between agents.

US-08 | Handle Errors and Demonstrate Workflow

Owner: Shared — all members

User story: As a user, I want the application to handle execution errors and display appropriate results so that I can use the multi-agent assistant reliably.

Acceptance criteria

- Missing input is handled.
- Search API failures are handled.
- Empty research does not produce fabricated summaries.
- LLM/API failures are handled appropriately.
- Invalid agent outputs are detected.
- File-generation failures are reported.
- Successful execution produces the notification text file.
- The workflow can be demonstrated from user input to final output.

Implementation tasks

- T-43: Implement workflow-level exception handling.
- T-44: Add meaningful execution logs.
- T-45: Test individual agents independently.
- T-46: Perform end-to-end integration testing.
- T-47: Test failure scenarios and recovery behavior.
- T-48: Prepare sample test cases and demonstration.
- T-49: Prepare project documentation and setup instructions.

---

05 / AGENT INPUT AND OUTPUT CONTRACTS

All developers should agree on a common data contract before integration. These examples are illustrative Python dictionaries; field names and types should remain consistent across the modules.

Agent 1 → Agent 2 | Research output

```
research_output = {
    "topic": "Generative AI",
    "findings": [{
        "title": "Technology development title",
        "description": "Research finding description",
        "source_url": "https://example.com/article",
        "published_date": "2026-11-10"
    }],
    "status": "success",
    "error": None
}
```

Agent 2 → Agent 3 | Summarizer output

```
summary_output = {
    "topic": "Generative AI",
    "key_points": [{
        "point": 1,
        "summary": "First key development",
        "sources": ["https://example.com/article1"]
    }],
    "status": "success",
    "error": None
}
```

The shortened example illustrates the schema only. Successful output must contain exactly five key points when sufficient valid research is available.

Agent 3 | Notification output

```
email_output = {
    "subject": "Technology Update: Generative AI",
    "body": "Dear Team,\n\nHere are the latest updates...",
    "file_path": "output/technology_update.txt",
    "status": "success",
    "error": None
}
```

Integration rules

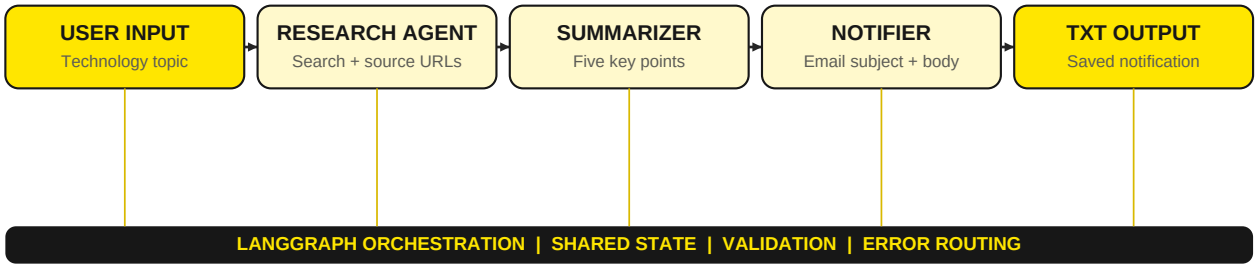
- Use consistent field names, data types, and status/error conventions.
- Keep source URLs attached to the findings and summary points they support.
- Validate structure at each agent boundary.
- Do not pass failed or empty output to the next agent as successful output.
- Treat publication date as optional.

06 / SOLUTION ARCHITECTURE AND LANGGRAPH FLOW

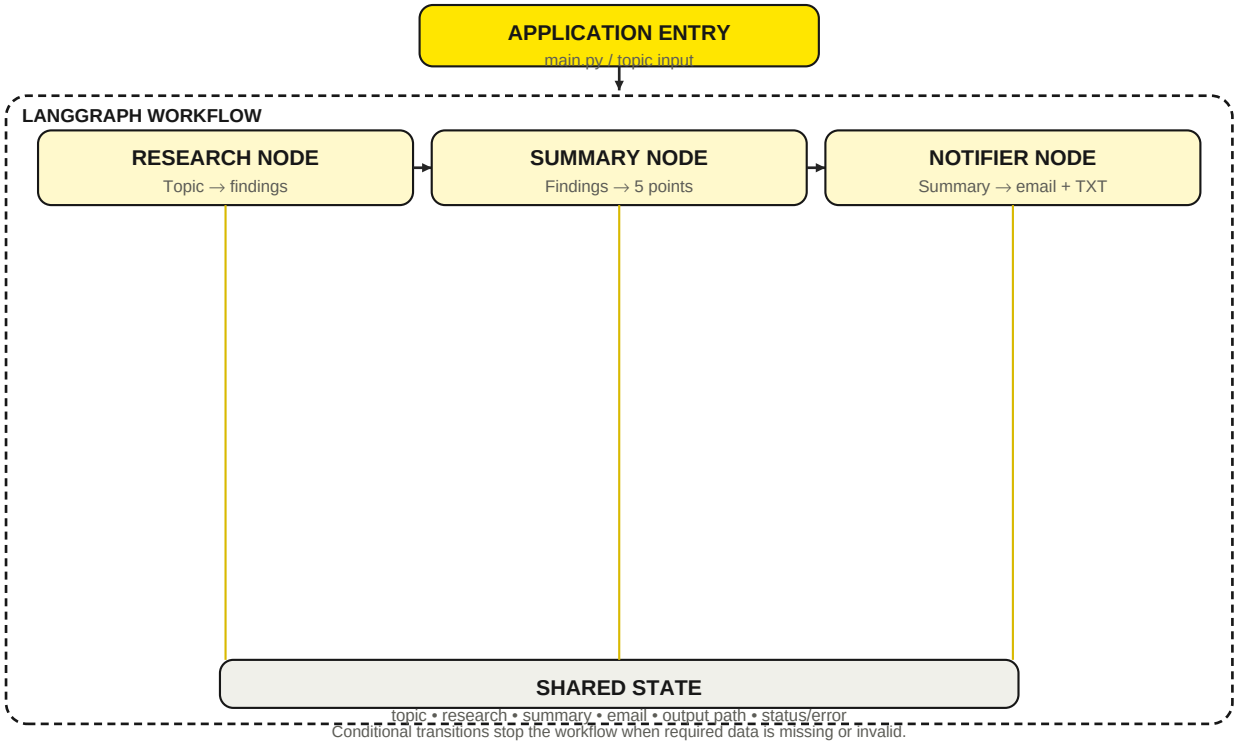
The application is organized into an entry point, a LangGraph workflow, three agent nodes, and a shared state. Each agent remains independently testable. The graph manages sequencing and routes execution to an error outcome when a required stage fails.

End-to-end workflow

End-to-end agent workflow



Architecture view



Shared state — suggested schema

```
from typing import TypedDict, Optional

class NewsAssistantState(TypedDict):
    topic: str
    research_results: list[dict]
    summary: list[dict]
    email_subject: Optional[str]
    email_body: Optional[str]
    output_file: Optional[str]
    status: str
    error: Optional[str]
```

Node responsibilities

NODE	RESPONSIBILITY
Input / Validation	Validate that the user supplied a non-empty topic.
Research	Call Agent 1 and store structured findings in shared state.
Summarization	Call Agent 2 and store five validated points and references.
Notification	Call Agent 3 to create email content and save the text file.
End / Result	Return completion status, error details, and output file location.

Failure and conditional routing

- Invalid input: stop before research and return a validation message.
- Search failure or no usable findings: stop before summarization.
- Invalid or insufficient summary: do not generate an email that claims to contain five verified points.
- LLM/API failure: record the failing stage and return a clear error.
- File-writing failure: report the file error and do not mark the workflow successful.

Retries should be limited to transient failures and must not create duplicate output files. Logs should identify the failing stage without exposing API keys or secrets.

07 / NON-FUNCTIONAL REQUIREMENTS

ID	REQUIREMENT	DESCRIPTION
NFR-01	**Reliability**	Handle API, agent, and file-system failures gracefully.
NFR-02	**Performance**	Avoid unnecessary repeated API calls and excessive processing.
NFR-03	**Maintainability**	Implement each agent as an independent Python module.
NFR-04	**Scalability**	Allow additional agents or workflow stages to be added later.
NFR-05	**Security**	Store API keys in environment variables; never hardcode or commit secrets.
NFR-06	**Data integrity**	Preserve source references through all workflow stages.
NFR-07	**Usability**	Show clear execution status and actionable error messages.
NFR-08	**Accuracy**	Ground summaries in collected findings and avoid unsupported claims.

Suggested project folder structure

```
multi_agent_news_assistant/
■■■ agents/
■   ■■■ research_agent.py
■   ■■■ summarizer_agent.py
■   ■■■ notifier_agent.py
■■■ orchestration/
■   ■■■ state.py
■   ■■■ graph.py
■■■ output/
■   ■■■ technology_update.txt
■■■ tests/
■   ■■■ test_research.py
■   ■■■ test_summarizer.py
■   ■■■ test_notifier.py
■   ■■■ test_workflow.py
■■■ .env
■■■ .env.example
■■■ .gitignore
■■■ requirements.txt
■■■ main.py
■■■ README.md
```

- **agents/:** Independent implementation of each specialist agent.
- **orchestration/:** Shared state definition and LangGraph workflow.
- **output/:** Generated notification files.
- **tests/:** Unit tests and end-to-end workflow tests.

- **.env:** Local secrets; must not be committed.
- **.env.example:** Required environment variable names without real values.
- **requirements.txt:** Shared Python dependencies.
- **main.py:** Application entry point.

08 / SPRINT PLAN, TESTING AND DEFINITION OF DONE

Proposed two-sprint plan

SPRINT	FOCUS	PLANNED WORK	EXIT CRITERIA
Sprint 1 Week 1	Individual agent development	Sreedev: research. Vishal: summarization. Vivek: email and TXT saving. All: agree on data contracts and dependencies.	Each agent runs independently with sample input and returns the agreed output structure.
Sprint 2 Week 2	Integration and testing	Integrate with LangGraph, implement shared state and failure paths, test data transfer, run end-to-end tests, prepare demo and README.	Valid topic flows through all agents and creates the TXT file; failures stop safely with clear messages.

Test scenarios

TEST ID	SCENARIO	EXPECTED RESULT
TC-01	Enter a valid topic	Input is accepted and research starts.
TC-02	Enter an empty topic	Validation error; no search is performed.
TC-03	Search returns multiple results	Relevant findings and source URLs are collected.
TC-04	Search API fails	Research-stage error; downstream agents do not run.
TC-05	No usable research	No fabricated summary is produced.
TC-06	Sufficient valid research	Exactly five grounded summary points are generated.
TC-07	Duplicate source URLs	Duplicate references are removed or deduplicated.
TC-08	LLM API fails	A clear agent error is returned.
TC-09	Valid five-point summary	Professional email subject and body are generated.
TC-10	Email generation succeeds	TXT file is created with subject and body.
TC-11	Output directory missing	Directory is created if permitted; file is saved.
TC-12	File-writing fails	File-stage error; workflow is not marked successful.
TC-13	Complete valid workflow	Notification file is generated and its path is returned.

User Story Definition of Done

- All implementation tasks for the story are completed.
- The functionality satisfies the acceptance criteria.
- Code is committed to the shared repository and reviewed according to team practice.
- The component has been independently tested.
- Input/output contracts are maintained.
- No hardcoded credentials or committed secrets are present.
- Relevant errors are handled and the owner has verified the implementation.

Epic Definition of Done

- All three agents work independently and are integrated through LangGraph.
- User input is accepted and validated.
- The Research Agent retrieves relevant information and source links.
- The Summarizer generates exactly five grounded points when sufficient research exists.
- Source links are preserved in the summary and email.

- The Notifier generates a professional email and saves it as a .txt file.
- Error-handling scenarios and end-to-end workflow are tested.
- README, setup instructions, and final demonstration are available.

09 / OWNERSHIP, AGILE BOARD AND SUCCESS CRITERIA

TEAM MEMBER	RESPONSIBILITY	TASK IDS	DELIVERABLE
Sreedev CS	Web Research Agent	T-01 to T-10	Research module and tests
Vishal Kumar Lal	AI Summarization Agent	T-11 to T-22	Summarizer module and tests
Vivek Sharma	Email Notification Agent	T-23 to T-34	Notifier module, TXT output, and tests
All members	Integration, testing, documentation	T-35 to T-49	LangGraph workflow, tests, README, demo

Recommended Agile issue hierarchy

- **Epic:** EPIC-01 — Multi-Agent AI News Assistant.
- **Features:** F-01 Web Research Agent; F-02 AI Summarization Agent; F-03 Email Notification Agent; F-04 Multi-Agent Orchestration & Integration.
- **User stories:** US-01 through US-08 under their corresponding features.
- **Sub-tasks:** T-01 through T-49 assigned to the responsible developer or shared team.

Suggested board statuses

TO DO → IN PROGRESS → IN REVIEW → TESTING → DONE

Team coordination guidelines

- **Independent development:** Each developer builds and tests their agent before integration.
- **Common interface:** Agree on field names, types, and error conventions before integration.
- **Shared environment:** Maintain a common requirements.txt and .env.example.
- **Version control:** Use separate feature branches and reviewed pull requests.
- **Integration responsibility:** All members participate; appoint one coordinator for the graph entry point and shared state.
- **Separation of concerns:** Research gathers information; summarization interprets findings; notification formats and saves the email.
- **Testing before merge:** Each agent must pass independent tests before integration.
- **Credential safety:** Keep .env out of version control and never expose keys in logs.

Final success criteria

The project is successful when a user enters a technology topic and the application executes the complete workflow: the Research Agent gathers recent developments from multiple sources; the Summarizer produces five concise, evidence-grounded points with source links; the Notifier creates a professional employee email; and the system saves that notification as a .txt file. LangGraph manages execution order, shared state, and failure paths.

EPIC COMPLETION OUTCOME

One topic in. Three specialized agents. Five key insights. One professional employee notification saved as a text file.